
Understanding Representation Bottlenecks in Neural Image-to-Text Models through Cross-Architecture Analysis

Rohan Anthony Siyuan Wang Trisha Sharma Cirenyangzhen

University of Bristol

EMATM0067 - Introduction to AI and Text Analytics, Team 10, Task 5

Code: <https://github.com/uob-aita-team10/team10-index>

Abstract

Describing images in natural language is a fundamental challenge in AI. In practice, it is difficult to analyse where and why certain approaches fail due to the complexity in real-world images. We study this problem in a synthetic setting, using simple geometric scenes with a closed vocabulary, to trace specific failures to design choices. We evaluate four architectures: a parallel classification model, a spatial attention LSTM, a Transformer decoder, and a controlled comparison of text representations. Across our experiments, we vary encoder training, spatial feature structure, decoder architecture, and text representation. The main pattern is that performance heavily depends on encoder adaptation. Fine-tuning improves exact match from 51% to 99.73% in the parallel classification model and from 47% to 88% in the Transformer. Preserving spatial structure and applying attention produces an 82 percentage point gain (15% to 97%) over global pooling. Text representation, on the other hand, has limited impact. Pretrained GloVe embeddings underperform randomly initialised embeddings, and extended training improves performance only to 19.5% under a frozen backbone. A zero-shot evaluation of GPT-4o mini achieves 5.4% exact match, far below task-specific models. The results point to the visual representation as the main bottleneck, with decoder design becoming secondary once this is addressed.

1 Introduction

Image captioning, the task of mapping visual input to a natural language description, is a fundamental challenge in AI. However, given the complexity of real-world images, it is often difficult to understand why certain approaches perform better than others. Too many variables change for individual factors to be cleanly isolated.

To investigate this, we use a controlled synthetic setting, training multiple architectures on simple geometric scenes to isolate specific failure modes.

The goal is to understand what actually drives performance in this setting. The synthetic setup allows this because the vocabulary is closed and the mapping from image to caption is deterministic, so errors can be attributed to specific design choices rather than ambiguity in the data. We exploit this to run controlled experiments across four approaches, varying encoder training, decoder architecture, spatial feature structure, and text representation to isolate the effect of each factor.

The base dataset consists of 2,000 synthetically generated 128×128 images across three scene types: 500 single-object, 1,000 binary two-object, and 500 triple three-object scenes. Binary scenes are generated from a fixed vocabulary of sizes, colours, shapes and spatial relations, with a single sentence template of the form "*a [size] [colour] [shape] is [relation] a [size] [colour] [shape]*". The dataset is

split 80/10/10 into train, validation and test sets. Larger datasets generated using the same procedure are used in specific experiments and are described below.

A correct prediction requires all attributes to be identified simultaneously: sizes, colours, shapes, and spatial relation. Exact match is used as the metric, since a single incorrect attribute makes the entire caption wrong. Size prediction is consistently the most difficult part of the task, as rendering at 128×128 limits the visual separation between size categories.

Our contributions are as follows:

- A synthetic dataset of 2,000 labelled scenes across three complexity levels, with a controlled vocabulary.
- A five-stage ablation on a parallel classification architecture separating the effects of training data scale and encoder fine-tuning.
- A spatial attention LSTM isolating the contribution of preserved spatial feature maps to relational reasoning.
- A Transformer decoder with patch-level cross-attention as a third independent test of encoder adaptation effects.
- A controlled comparison of text representations: random embeddings, one-hot encoding, and GloVe vectors.
- A zero-shot evaluation of GPT-4o mini as an external calibration against commercial LLMs.

Across these experiments, we compare how different design choices affect performance and where failures arise.

2 Methods

2.1 Dataset and Scene Generation

The base dataset consists of synthetically generated scenes containing simple geometric objects. Each object is defined by a set of attributes: three sizes, eight colours, eight shapes, and eight spatial relations. Binary scenes follow a fixed sentence template of the form “*a [size] [colour] [shape] is [relation] a [size] [colour] [shape]*”. This ensures a closed vocabulary and a deterministic mapping between image and description, so any prediction error can be traced back to a specific attribute rather than ambiguity in the data.

We also use extended versions of the dataset in later experiments. For the encoder ablation, the binary dataset is expanded to approximately 12,000 samples to isolate the effect of data scale under both frozen and fine-tuned backbones. A revised dataset generation pipeline introduces greater variation in caption structure and visual appearance. This version incorporates randomised background colours to increase visual variation. These extended datasets are used to test whether increasing data diversity changes the performance trends observed earlier.

A known challenge in the dataset arises from size discrimination. All images are rendered at a resolution of 128×128 using marker-based plotting, which limits the visual separation between size categories. As a result, differences between “small”, “medium”, and “large” objects can be subtle. This is a limitation of the dataset caused by the rendering process, not necessarily a failure of the models themselves.

2.2 Baseline: Parallel Classification Architecture

The baseline model is designed around a simple question: should caption generation be treated as a sequential prediction problem, or as a set of independent attribute predictions? In this task, each caption encodes seven attributes: two sizes, two colours, two shapes, and one spatial relation. These attributes are structurally parallel rather than sequential, so predicting them independently is likely a better fit than generating tokens step-by-step. The idea is that removing unnecessary sequential dependence should make the learning problem simpler and improve accuracy. A limitation of this approach is that it only works for binary scenes, unless the number of classification heads is adjusted dynamically.

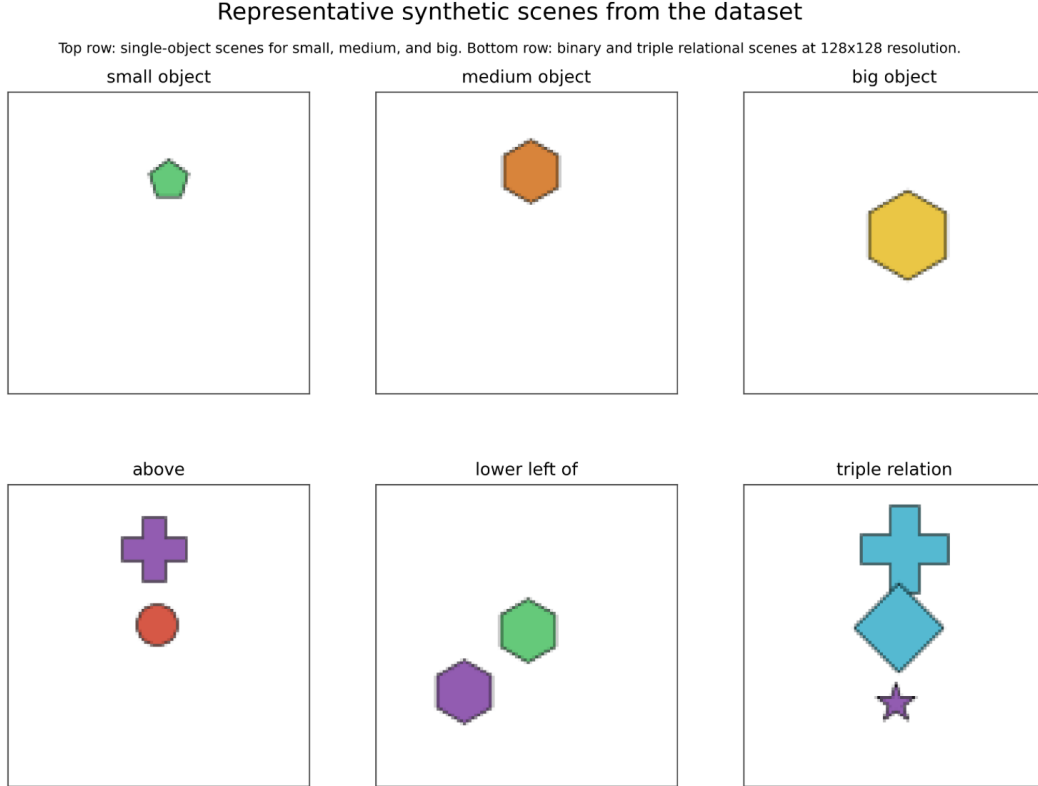


Figure 1: Example synthetic scenes illustrating variation in object attributes.

The model uses a ResNet18 encoder He et al. [2016] to extract a global visual representation, followed by seven independent classification heads. Each head corresponds to one attribute: size, colour, and shape for each of the two objects, and a single head for the spatial relation. Each head is implemented as a two-layer MLP consisting of a linear projection ($256 \rightarrow 128$), followed by ReLU activation and dropout (0.3), and a final linear layer mapping to the corresponding vocabulary.

Training is done under two conditions: with the encoder frozen and with it fine-tuned. This is the main axis of the ablation in the next section. All other components are kept fixed so the effect of encoder adaptation can be isolated.

2.3 Axis 1: Encoder Fine-Tuning: Five-Stage Ablation

This experiment focuses on how much encoder adaptation actually matters, using a five-stage ablation. We vary two factors independently: dataset size and whether the backbone is frozen. The key question is whether the plateau under a frozen encoder is due to limited data, or whether it reflects a deeper issue with using ImageNet-pretrained features Deng et al. [2009] on synthetic geometric scenes.

Across the five stages, we separate the effects of data scale and encoder adaptation. The earlier stages increase dataset size while keeping the backbone frozen. Stage 5 then introduces end-to-end fine-tuning on the full dataset. This makes it easier to see which factor is actually driving performance.

2.4 Axis 2: Spatial Feature Structure: Attention LSTM

The second axis focuses on the impact of introducing spatial information on performance, as the description in this task is based on object and positional relationships (above, below, left, right). This information is difficult to capture if the image is compressed into a single global feature vector. The Attention LSTM architecture uses ResNet18 as the encoder, and the final feature map is adaptively pooled into a 7×7 spatial grid, generating 49 feature vectors corresponding to different regions. The decoder is a recurrent decoder based on LSTMCell Hochreiter and Schmidhuber [1997] with

an additive attention mechanism Bahdanau et al. [2015]: at each time step, attention weights are computed over the 49 feature vectors using the current hidden state, thereby generating a weighted visual context vector. This allows the model to focus on different regions of the image when generating descriptions of different parts, incorporating spatial information. This is compared with a baseline LSTM model using a single globally pooled ResNet18 feature vector to isolate the contribution of spatial attention. A third variant using CBAM Woo et al. [2018] is also evaluated, applying channel and spatial attention gates within the encoder as an intermediate condition between global pooling and full spatial attention.

2.5 Axis 3: Decoder Architecture: Transformer with Patch Cross-Attention

The third axis explores an alternative decoding paradigm using a Transformer decoder Vaswani et al. [2017] with cross-attention over spatial features. The model attends to 49 spatial patches at each decoding layer, without relying on a recurrent hidden state. The hypothesis is that patch-level cross-attention provides richer and more direct access to visual information at each step of generation.

A practical challenge in this setup is computational cost. To address this, features from the encoder are precomputed and cached when the backbone is frozen. This significantly reduces training time and makes large-scale experiments feasible on limited hardware. This design choice enables a direct comparison between frozen and unfrozen backbones without requiring repeated forward passes through the encoder.

As with the baseline model, the primary variable of interest is whether the encoder is frozen or fine-tuned. The Transformer architecture serves as a third independent test of the role of encoder adaptation, as opposed to a direct competitor to the other decoders.

2.6 Axis 4: Text Representation

The final axis examines the role of text representation in the task. Three alternatives are compared using a separately implemented model with a frozen MobileNetV2 backbone Sandler et al. [2018]: randomly initialised embeddings, one-hot encoding with linear projection, and pretrained GloVe vectors Pennington et al. [2014]. This axis is motivated by the fact that, although the task is visually grounded, the decoder must still learn a mapping from visual features to token sequences, and different token representations may affect how easily this mapping is learned.

We hypothesise that random embeddings may already be sufficient in this small, highly regular vocabulary setting. One-hot representations may suit the discrete attribute structure of the task, while GloVe may offer limited benefit because the synthetic captions do not reflect natural language co-occurrence patterns.

This axis tests whether language-side improvements can compensate for limitations in the visual encoder. If performance remains limited across text representations, this would suggest that limitations may lie in the visual representation rather than the decoder or token encoding. The comparison therefore serves as a controlled test of how far language-side modifications alone can improve performance in this task.

2.7 LLM Comparison: GPT-4o mini

A zero-shot evaluation using GPT-4o mini OpenAI [2024] is included as a commercial LLM reference point. The goal is not a direct comparison with task-specific models, but to provide a rough calibration of task difficulty and the role of supervised training. A subset of 130 images is evaluated using a fixed prompt, and outputs are parsed into the same structured format used for model evaluation.

This helps put the results in context. If a general-purpose model performs poorly, it suggests that the task requires precise grounding in visual features that cannot be recovered from language priors alone. Conversely, strong performance would indicate that the task is largely solvable through general reasoning.

3 Experimental Evaluation

3.1 Metrics and Their Limitations

We use exact match as the primary evaluation metric across all experiments. A prediction is only counted as correct if all seven attribute slots (size_1 , colour_1 , shape_1 , relation , size_2 , colour_2 , shape_2) are correct at the same time. This choice follows directly from the task structure. Captions are compositional and deterministic, so a prediction that identifies six of seven attributes but misidentifies one, produces a caption that is factually wrong. Giving partial credit would hide these errors instead of exposing them.

Per-slot accuracy is used as a supplementary diagnostic. Exact match alone does not show where errors come from, so we also report per-slot accuracy. This decomposition is particularly useful for identifying systematic weaknesses, for instance, whether size errors are more common than relation errors, or whether a particular architecture struggles with a specific slot.

Sequence-level metrics such as BLEU Papineni et al. [2002] are not used. BLEU measures n-gram overlap between generated and reference strings, which is appropriate when outputs vary in length and phrasing. In this task, all captions share a fixed seven-attribute template, so n-gram overlap between a wrong and correct caption can be high even when multiple attributes are wrong. This would give a misleading view of performance on a closed-vocabulary compositional task.

3.2 Experimental Setup

All experiments use an 80/10/10 train/validation/test split. The base dataset of 2,000 samples provides test sets of approximately 100 binary scenes. Extended datasets are used for the encoder ablation and Transformer experiments. These are generated using the same rendering pipeline but with larger binary sample counts. In these cases, the test set scales proportionally, yielding approximately 1,494 binary test scenes for the 15,000-sample dataset.

The encoder ablation (Section 3.3) trains for 30 epochs in Stages 1–3, 50 epochs in Stage 4, and until convergence in Stage 5. We use the Adam optimiser with a learning rate of 10^{-3} for the classification heads. When the backbone is unfrozen, a lower learning rate of 10^{-4} is applied to backbone parameters. Batch size is 64 throughout.

The spatial attention LSTM and Transformer experiments also use Adam, with learning rates in the range 10^{-3} to 10^{-4} depending on the setup. Text representation experiments are evaluated at 3 and 30 epochs under a frozen backbone. All experiments are implemented in PyTorch. Pretrained ResNet18 weights are loaded via `torchvision.models`; MobileNet weights are loaded from the same source. GPT-4o mini is accessed via the OpenAI API with a fixed structured prompt.

3.3 Results: Encoder Fine-Tuning Ablation

The plateau under frozen features. Stages 1 through 4 keep the backbone frozen while increasing the number of binary training samples from 801 to 12,012 and extending training from 30 to 50 epochs. Table 1 reports exact match accuracy across all five stages. Performance improves with data from 13.13% at 801 samples to 48.53% at 12,012 samples but plateaus well below ceiling. Extending training from 30 to 50 epochs at the same data scale (Stages 3 and 4) yields only a marginal gain of 2.5 percentage points (48.53% to 51.00%). This suggests the model had already converged rather than being undertrained.

The fine-tuning breakthrough. Stage 5 introduces end-to-end fine-tuning with a differential learning rate: backbone parameters are updated at $0.1 \times$ the head learning rate to prevent destructive interference with pretrained representations. The model reaches 99.73% exact match (1,490 of 1,494 test scenes correct) and converges in six epochs. The speed of convergence is also informative: the model does not require large numbers of gradient steps once the backbone is free to adapt. The conclusion is that the bottleneck across Stages 1–4 was encoder adaptation, not data volume or decoder capacity. Frozen ImageNet features, trained on natural images, do not represent the flat-colour synthetic geometry of the scenes well. Fine-tuning removes this constraint entirely.

Table 1: Five-stage encoder ablation. Stage 1 is evaluated on approximately 99 binary test scenes, Stage 2 on approximately 499, and Stages 3–5 on 1,494 scenes, corresponding to the 80/10/10 split of their respective datasets.

Stage	Train samples	Backbone	Epochs	Exact match
1	801	frozen	30	13.13%
2	4,007	frozen	30	25.65%
3	12,012	frozen	30	48.53%
4	12,012	frozen	50	51.00%
5	12,012	fine-tuned	6	99.73%

Residual errors and size slot concentration. The four remaining failures at Stage 5 are attributable to size misclassification in three cases (size_1 predicted incorrectly in two instances, size_2 in one) and a single relation error (lower-left-of predicted as above). Per-slot accuracy at Stage 5 is reported in Table 2. Colour, shape, and spatial relation slots all achieve at or near 100%; size slots are the sole source of systematic error. This pattern is consistent across all five stages: size_1 is the weakest slot at every stage of the ablation. This is not a model capacity issue, but a limitation of the rendering. At 128×128, visual separability between size categories is limited, and the difficulty comes from the input itself rather than something the model can fix architecturally.

Table 2: Per-slot accuracy at Stage 5 (fine-tuned, 12,012 training samples). Test set: 1,494 binary scenes.

Slot	Correct / Total	Accuracy
size_1	1,492 / 1,494	99.87%
colour_1	1,494 / 1,494	100.00%
shape_1	1,494 / 1,494	100.00%
relation	1,493 / 1,494	99.93%
size_2	1,493 / 1,494	99.93%
colour_2	1,494 / 1,494	100.00%
shape_2	1,494 / 1,494	100.00%

3.4 Results: Spatial Attention LSTM

Spatial information experiments compared the global pooling ResNet18-LSTM baseline model with an attention-based LSTM model using 7×7 spatial feature maps. The global pooling baseline model only achieved 15% greedy sentence exact matching and 21% beam search sentence exact matching, indicating that beam search offers limited benefits when visual feature representation is insufficient. Meanwhile, the attention-based LSTM model achieved 97% greedy sentence exact matching and beam search sentence exact matching, with the validation loss decreasing from 0.4728 (baseline) to 0.0127. This improvement supports the hypothesis that spatial information is important for describing positional relationships. A single global pooling feature vector compresses object attributes and spatial layout into a single feature vector, ignoring positional relationships such as top, bottom, left, and right, ultimately creating a performance bottleneck. In addition, a Convolutional Block Attention Module (CBAM) Woo et al. [2018] variant is evaluated as an intermediate condition. CBAM applies channel and spatial attention directly within the encoder feature maps, without introducing an explicit attention mechanism in the decoder. Table 3 summarises the Group A results.

Table 3: Group A: core architecture comparison (ResNet18 encoder, all runs fine-tuned end-to-end).

Run	Setup	Val Loss	Greedy Exact	Beam Exact
A1	LSTM (global pool)	0.4728	0.15	0.21
A2	LSTM + CBAM	0.4592	0.17	0.22
A3	Attention-LSTM	0.0127	0.97	0.97

CBAM provides only a small improvement over global pooling (A2 vs A1), suggesting that applying attention within the encoder is not enough. The decoder still requires access to explicit spatial feature maps, rather than features that have only been reweighted.

Convergence with the encoder ablation finding. These two experiments fail for different reasons, but point to the same issue. In the encoder ablation, the problem is that frozen ImageNet features do not adapt to the synthetic domain. In the spatial attention setup, the issue is different: collapsing everything into a single vector removes the spatial information needed for relations. In both cases, the limitation appears before the decoder, in the information available to it. Once that is fixed, both approaches work well.

3.5 Results: Transformer Decoder

Frozen versus unfrozen backbone. The Transformer decoder experiment tests a third decoding paradigm: cross-attention over 49 spatial patches at each decoding layer, with no recurrent hidden state. Under a frozen backbone, exact match is 47%. With end-to-end fine-tuning, this rises to 88%. The pattern is consistent with both previous experiments: backbone adaptation is the primary driver of performance, and the frozen-to-fine-tuned gap is large regardless of decoder architecture. Training under a frozen backbone completed 30 epochs in under 5 minutes (10 seconds per epoch), but produced systematic colour confusion, most notably “cyan” predicted as “blue”, which is consistent with frozen ImageNet features being unable to distinguish perceptually similar flat synthetic colours. With the backbone unfrozen, training took approximately 85 seconds per epoch over 50 epochs, and colour confusions are eliminated entirely, confirming that backbone adaptation is the decisive factor in resolving fine-grained colour discrimination. Table 4 reports results for both configurations.

Table 4: Transformer decoder results on 1,600 training samples. Backbone fine-tuning is the primary variable.

Configuration	Backbone	Exact Match	Token Accuracy
Frozen backbone	frozen	47.00%	88.80%
Unfrozen backbone	fine-tuned	88.00%	98.41%

The gap from 88% to ceiling. The fine-tuned Transformer reaches 88%, below the 97% and 99.73% ceilings achieved by the spatial attention LSTM and parallel classification model respectively. This gap is attributable to dataset scale rather than decoder architecture. The Transformer experiment uses the 2,000-sample base dataset; the other two experiments use extended datasets of 12,000–15,000+ binary training samples. The relationship between data scale and performance observed in the encoder ablation (Section 3.3) predicts that a fine-tuned Transformer trained on the larger dataset would approach a similar ceiling, though this remains untested at that scale. The 88% result should therefore not be interpreted as a decoder failure but as a data scale effect consistent with the broader experimental pattern. Once the encoder is allowed to adapt, differences between decoder architectures become secondary compared to representation quality and data scale.

A third independent confirmation. The Transformer setup differs substantially from the previous approaches, replacing both parallel output heads and recurrent state with self-attention and cross-attention over spatial patches. Despite this change, the overall behaviour remains consistent: most of the performance gain comes from allowing the encoder to adapt. Once the visual features are sufficiently informative, further changes to the decoder have a comparatively small effect. This makes the result less about the specific architecture and more about what information is available to it.

3.6 Results: Text Representation

Ordering of text representations. Three text representation strategies are evaluated under a frozen MobileNet backbone: randomly initialised embeddings, one-hot encoding, and pretrained GloVe vectors. Table 5 reports exact match and token accuracy across all three representations. At 3 epochs, exact match accuracy is 6.0% for random embeddings, 2.5% for one-hot, and 1.0% for GloVe. Random embeddings also achieve the highest token accuracy at 62.64%. The ordering suggests that

pretrained or more structured language representations do not automatically improve performance in this setting.

Table 5: Text representation comparison (MobileNetV2 encoder, frozen, 3 epochs). Exact match on binary test scenes.

Representation	Exact Match	Token Accuracy
Random embeddings	6.0%	62.64%
One-hot + projection	2.5%	47.36%
GloVe (100d)	1.0%	55.68%

Why richer representations do not help. GloVe vectors are learned from natural language corpora, where words are organised by distributional co-occurrence. In this task, however, the vocabulary is small, closed, and synthetic, and the captions follow fixed compositional templates. The semantic structure encoded by GloVe therefore does not transfer cleanly. Random embeddings, by contrast, are free to organise the vocabulary according to the task-specific distinctions required by the visual prediction problem. One-hot representations preserve the discrete attribute structure of the task, but appear less efficient than a learned continuous embedding space.

Extended training. To test whether the 3-epoch comparison reflects convergence or simply undertraining, random embeddings were trained for 30 epochs with learning-rate decay. Exact match improved to 19.5%, with the best checkpoint at epoch 26. This shows that the low 3-epoch result partly reflects an undertrained decoder rather than a hard ceiling imposed by the representation itself.

The binding constraint. Even after extended training, the best text representation remains far below the performance achieved by models with stronger visual representations in the other experimental axes. This experiment therefore acts as a control: language-side improvements affect learning efficiency, but they do not compensate for limitations in the visual encoder. This indicates that improving the decoder alone is not sufficient when the visual features are limited.

3.7 LLM Comparison: GPT-4o mini

We run a zero-shot evaluation of GPT-4o mini on 130 images using a fixed prompt. The model is asked to describe each scene in the same seven-attribute format used throughout this work. The outputs are then parsed into slots and compared against ground truth using exact match.

The model achieves 5.4% exact match (7/130). Looking at individual attributes, colour performs best at 85.4%, followed by shape at 73.1%. Size is much weaker at 26.9%, which is below the 33.3% expected from random guessing across three categories. This suggests the model is not reliably distinguishing size at all. We also see vocabulary mismatches: nine responses use out-of-vocabulary terms (e.g. “square” instead of “cube”), which indicates misalignment with the closed label set.

Overall, the gap to the task-specific models is large. Fine-tuned models reach 88%–99.73%, while GPT-4o mini remains at 5.4%. Colour and shape transfer reasonably well, but size is consistently the weakest attribute, which matches the same failure pattern seen earlier. This suggests the model struggles to extract the relevant signals without adaptation to the rendering setup.

3.8 Convergent Error Analysis

Size as a universal failure mode. Across all experiments, size is the hardest attribute to predict. In the encoder ablation, it is the weakest slot at every stage, and three of the four remaining errors at 99.73% come from size. The same pattern shows up in the Transformer and spatial attention models, where size accounts for a large share of the remaining mistakes. GPT-4o mini achieves 26.9% accuracy on size, which is below the 33.3% expected from random guessing across three balanced classes. This reflects the same failure mode observed in the trained models. As a general-purpose model with no exposure to the rendering setup, it is unable to reliably recover size distinctions in this setting.

Another issue under frozen backbones is colour confusion. In the Transformer setup, similar colours such as “cyan” are often predicted as “blue”. This disappears once the backbone is fine-tuned, which suggests the pretrained features are not well suited to the flat synthetic colours in the dataset.

The size problem is different. At 128×128 resolution, the visual difference between size categories is small. The model has to pick up on subtle changes in marker size, which are not always clearly separated. Because of this, errors in size persist even after changing the architecture, and only reduce once the encoder is fine-tuned and enough data is available.

Spatial relation errors and their conditions. Relation errors are rare once the model is properly trained. Both the attention LSTM and the parallel classification model reach above 99% accuracy on relations, with only a single mistake in the best setting.

These errors become more common when the backbone is frozen or when data is limited. In the spatial attention experiment, the global pooling baseline performs poorly (15% exact match), which suggests that collapsing the image into a single vector removes the information needed to recover spatial relationships. In this case, the issue is not the decoder, but the loss of spatial structure in the visual features.

Across all four experimental axes, the failure modes are different in form but similar in cause: the visual representation fails to encode the distinctions required by the task. In the encoder ablation, this appears as domain mismatch, where ImageNet features do not represent flat synthetic geometry. In the spatial attention setup, the issue is not mismatch but information loss, where global pooling removes the spatial structure needed for relational reasoning. In the Transformer, the same domain mismatch reappears despite a different decoder. Finally, the text representation results show that improving the decoder or token space does not compensate when the visual input itself is insufficient. These are not separate problems but different manifestations of the same constraint: if the encoder does not preserve task-relevant structure, no downstream component can recover it.

4 Conclusions

The clearest takeaway from these experiments is how much everything depends on the encoder. When it is frozen, performance plateaus even with more data or training. Once it is fine-tuned, the jump is immediate. In the parallel classification model this goes from 51% to 99.73%, and in the Transformer from 47% to 88%. A similar effect appears in the spatial attention setup, where preserving spatial structure pushes performance from 15% to 97%.

Other changes matter far less by comparison. Increasing dataset size under a frozen backbone helps, but only up to a point. Changing the decoder also has limited impact if the visual representation is not already good. On the language side, GloVe embeddings do not help in this setting, and a zero-shot GPT-4o mini baseline remains far behind the trained models at 5.4% exact match.

All four approaches end up failing in different ways, but point to the same issue. In some cases the encoder is not adapted, in others spatial information is lost, and in others the representation is simply not rich enough. But in all of them, the limitation appears before the decoder. Once the visual features are good enough, the rest of the system becomes much easier to get right.

There are still some clear limitations. Size remains the hardest attribute to predict, mainly because the visual differences are small at 128×128 resolution. The dataset also uses a fixed sentence template, so it does not test variation in language. Triple-object scenes are not explored across all models, and the parallel classification setup does not extend naturally beyond binary scenes.

Overall, the results show that performance in this setting is determined primarily by the quality of the visual representation. Once the encoder is able to capture the relevant structure in the input, differences between decoders become secondary. This pattern holds consistently across all four experimental axes, despite their architectural differences.

References

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *ICLR*, 2015.

- Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CVPR*, 2016.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.
- OpenAI. Gpt-4o technical report, 2024. <https://openai.com>.
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *ACL*, 2002.
- Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *EMNLP*, 2014.
- Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *CVPR*, 2018.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 2017.
- Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. CBAM: Convolutional block attention module. In *ECCV*, 2018.